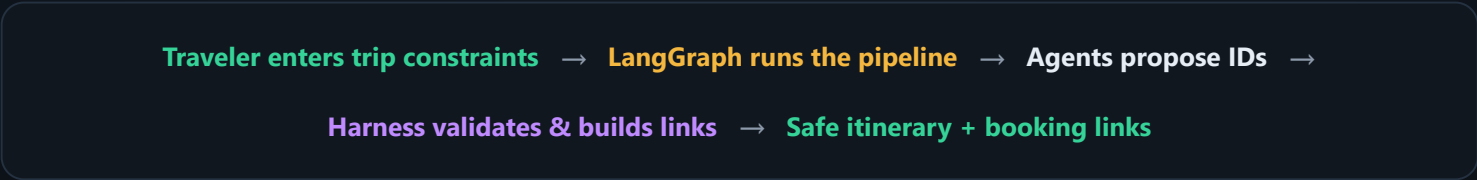


GitTrippin — System Architecture

How the pieces fit: from the traveler's request, through the untrusted agents, past the zero-trust harness, into persisted state.



LAYER 1

Front End

What the traveler touches. CLI for the demo; could be a web page later.

Command Line

The interface for the demo run.

run --user X --trip austin

replay <id> --from hotel

Inputs

A real trip profile: origin, dates, budget, min rating, diet, cuisine & activity likes.

CLI prompts

trip.json

Outputs

A bookable itinerary the traveler actually receives.

HTML itinerary

whitelisted links

cost summary

LAYER 2

Back End

The brain. Runs the stages in order and enforces the rules.

Orchestrator

LangGraph state machine runs 4 agent nodes, wraps every node with the harness, routes the result.

graph.py

StateGraph

Retry loop

On a failed check, replay from the last good checkpoint with the exact problem. Up to 3 tries.

replay_from_checkpoint

Human escalation

No option under budget, unverifiable booking, or 3 fails → stop & ask a person.

interrupt → HITL

↓ agents only propose IDs — they never build links or write state ↓

LAYER 3

AI Agents

Untrusted, swappable workers, all with one contract: run(input, feedback) → IDs.

Flight agent

Finds flights fitting dates & budget.

→ flight_id

Hotel agent

Lodging by location, budget & min rating; reasons a fallback over a bad booking.

→ hotel_id

Rental-car agent

Local transport logistics for the dates.

→ car_id

Experience agent

Localized activities & dining for the diet/cuisine.

template

claude (swap-in)

→ poi_ids

LAYER 4

Guardrails

Pydantic schemas in; IDs not URLs out; a deterministic Shadow Auditor

Checkpoints

Pass/fail gate after each node: valid schema, on

Material Handling

Routes only the fields each agent needs — food prefs never reach Flight/Hotel.

Alarms

Structured JSON when a rule trips — severity &

Zero-Trust Harness

The inspector. Sits in the gap between "proposed" and "booked".

regrades every claim (rating ≥ 3.5).

schemas.py

url_whitelist

shadow_auditor

budget, rating threshold met. State persists to replay.

checkpointer

3-strike → HITL

MaterialHandler.route

SCOPE

recommended action, streamed live.

type · severity · context · action

→ Splunk

LAYER 5

Data / Storage

The only layer that persists — LangGraph checkpoints + records.

Session memory

Remembers a traveler across runs: profile, trip history & feedback.

UserStore

JSON files

Checkpoint records

Every node's validated state saved by thread_id, so a run can be replayed, not restarted.

MemorySaver → Sqlite

./runs/<id>/

Seed data

Inventory the agents read from for the demo (no live network needed).

flights.py

hotels.py

Runtime

Containerized boundary — fixed, reproducible, no ambient trust for agents.

Docker

Tech: Python · LangGraph (state, checkpointing, replay) · Pydantic (schemas, sandboxing) · Docker · Splunk.

Key idea: the four pillars live outside every agent — validation, link-building, state & alarms are decoupled from the untrusted AI.